will run only the specs listed as failing in the examples.txt file—in this case bypassing the slower successful test, which is handy. You also have rspec --next-failure, which also runs the failing tests but stops after the first test that is still failing. RSpec guarantees that the run order will be the same for a series of successive rspec --next-failure runs, so this can be a good way to work through a bunch of failed tests.

Back to making the test pass. Add a validation to the Project model:

test_first/06/app/models/project.rb
```
validates :name, presence: true
```

Now, use that information in the workflow to determine failure. One thing I like to do in this kind of workflow is start with the assumption of failure and only show success when I know that the workflow has succeeded:

test_first/06/app/workflows/creates_project.rb
```
def initialize(name: "", task_string: "")
  @name = name
  @task_string = task_string || ""
  @success = false
end

def success?
  @success
end

def build
  self.project = Project.new(name: name)
  project.tasks = convert_string_to_tasks
  project
end

def create
  build
  result = project.save
  @success = result
end
```

Here an @success instance variable is added with an initial value of false. It is set to true only if the result of saving the project in the create method is true. This causes the workflow test to pass, because the no-name project will fail the validation, causing the result of saving to be false and causing success? of the workflow to also be false. Additionally, you have an easy mechanism to support other possible failure modes when they arise.

Next, update the controller to know about the possibility of failure: